Learning to Select Context per Code Diff: An Accuracy–Cost Trade-off for LLM Code Review

Michael Liu (msl196) **Ruiding Feng (rf711)** **Li-Hua Peng (lp880)**
Machine Learning II / CS536, Rutgers University

Abstract

We study whether selecting context for each code change can reduce the input cost of LLM-assisted code review without reducing accuracy. Our planned method trains a small context selector while keeping the reviewer fixed. So far, we have built a repository-based data split, a Qwen3B QLoRA training workflow, and paired evaluation tools. We have also implemented fixed, random, and similarity-based context policies. Much of our work has involved making cloud runs recoverable and checking that the evaluation compares the same examples. Paired model results are still being collected, so we cannot yet assess the benefit of adaptation or context selection.

Repository: `https://github.com/michaelsliu3/CS536_Project`

1 Research question

A code reviewer may need an issue description to understand a change's purpose or nearby code to see how it will be used. Giving the model every available field uses more tokens, even when some fields add little useful information. We ask: *Can a learned selector choose context for each code change while maintaining review accuracy and using fewer tokens than fixed context choices?*

We use ContextCRBench because it provides code changes together with issue, pull request (PR), and surrounding-code context [1]. Repoformer studies selective retrieval for code completion [2]; our question is whether a separate selector can help with code-review judgments. Following instructor feedback, we focus on this accuracy–cost trade-off. Robustness is outside the core study, and transfer to another reviewer is optional.

Our target is accuracy within one percentage point of the best fixed strategy with at least 30% fewer extra context tokens, or higher accuracy at a matched budget. We first built a Qwen3B adaptation pilot to work through the data and evaluation pipeline. That pilot asks whether fine-tuning improves merge-status prediction. The main experiment will instead keep Qwen frozen and train the selector.

2 Data and experimental setup

Task and split. The reviewer returns a short JSON answer with the Boolean field `is_merged`. Merge status is the benchmark target, but it is only a proxy for code quality: a merged change can contain a bug, and a rejected change can have nontechnical reasons. We exclude explicit outcome fields and human review answers from prompts. We still need to check whether issue and PR text contains later edits or quoted solutions.

Our split targets 80/10/10 training/validation/test proportions by repository, keeping related PR changes together. The proportions refer to repositories, so example counts can differ. The split code removes exact normalized-diff duplicates and excludes duplicate groups with conflicting labels. A saved manifest fixes membership and records exclusions. We use validation for development and

reserve the test repositories for the final comparison. Grouping by repository does not by itself rule out near-duplicates or temporal leakage.

A Python example. Training record `final_json/63859.json` is from `pandas-dev/pandas`, PR #53773. Its issue describes a `TypeError` when nested JSON contains `null`. The following four lines are copied from the diff; the header, function signature, and other lines are omitted. A leading + marks an addition.

```
+         if data is None:
+             return
          if isinstance(data, dict):
              data = [data]
```

The new guard returns before processing `None`. The dataset label is `False`, which records merge status rather than whether this guard is a correct fix. We show the example to explain the input format, not as evidence of model performance. The label is kept out of the prompt.

Training pilot. The notebook is configured for Qwen2.5-Coder-3B-Instruct with QLoRA [3]: 4-bit NF4, double quantization, adapter rank 16, scaling 32, and dropout 0.05. It uses a learning rate of 2×10^{-4}, a cosine schedule with 3% warmup, and weight decay 0.01. Batch size one with eight accumulation steps gives an effective batch size of eight on one GPU. The sequence cap is 2,048 tokens, and loss is computed on answer tokens. We need to confirm the completed run's settings and epoch count against its saved cloud artifacts.

Cloud checkpoints save the adapter, optimizer, scheduler, random state, and training progress. Configuration and logs are preserved across sessions. Restarting a run restores its saved epoch target; extending a completed run changes the learning-rate schedule and must be recorded as a continuation experiment.

Evaluation and context policies. The evaluation notebook compares the untouched base, the base with its trained adapter, and a constant prediction chosen from training-label counts. All three use the same saved validation sample, with up to 200 examples selected before inference. The two model conditions share prompts, truncation, quantization, and deterministic decoding. Invalid JSON counts as an error and is also reported separately. We will compare accuracy, macro-F1, per-class scores, and paired improvements and regressions.

For the frozen-reviewer study, we have implemented diff-only, all-context, fixed-mask, random, and BM25 similarity policies. They select among issue text, PR text, and code before and after the change. The selection code checks token budgets on assembled prompts and uses fixed seeds for random choices. These implementations still need to be evaluated with real reviewer outputs. The best fixed mask and token budgets will be chosen on validation data.

3 Progress and results still needed

The implementation now covers repository splits, checkpoint recovery, paired prediction exports, and offline metrics and error analysis. We tested the analysis tools on synthetic predictions; verified cloud outputs are still needed to fill Table 1. Dashes indicate missing measurements, not zero scores. A falling training loss alone would not tell us whether generated judgments have improved.

Table 1: Comparison to be completed on the same saved validation sample. The final test partition is reserved.

Method	Accuracy	Macro-F1	Invalid JSON rate
Training-majority constant	—	—	N/A
Untouched Qwen3B	—	—	—
Qwen3B + trained adapter	—	—	—

[To complete: Add the run/checkpoint identity, completed epochs, validation sample and class counts, Table 1 scores, and a training/validation loss plot before submission.]

4 What we learned

Cloud training required more attention to recovery than we initially expected. Model-download timeouts led us to load a local copy of the weights, and limited system-disk space required moving checkpoints to persistent storage. We also changed run matching so that harmless notebook edits do not block recovery, while changes to training settings select a different run directory.

Full validation was expensive enough to interrupt training frequently. We changed its interval from 100 to 1,000 optimizer steps while keeping frequent checkpoint saves. On resume, Trainer initially restored the older interval; the callback now reapplies the configured value. We still need timing measurements to quantify the effect of this change.

The data also needs careful interpretation. The Python example shows why merge status should not be read as a direct verdict on a fix. We need to inspect ambiguous labels and context timing before drawing conclusions. Long inputs are another concern: the 2,048-token cap can remove some context, so an all-context prompt does not guarantee that the reviewer sees every field. We will report truncation alongside the main metrics.

5 Next steps

Our immediate task is to recover the completed run's configuration and predictions, fill Table 1, and inspect both improvements and regressions. We will use those artifacts to check whether adaptation changed generated answers on held-out repositories. The pilot results will be reported separately from the context-selection experiment.

We will then run a small frozen-reviewer comparison of context policies. Before generating a larger outcome set, we need to see whether different context choices change enough judgments to make per-example selection useful. Reviewer outcomes from training repositories will provide targets for a compact transformer selector. We will compare its accuracy and extra context tokens with the fixed, random, and similarity baselines, including selector time in the runtime measurement. Repeated training seeds and context-type ablations are planned. If the selector does not beat the best fixed policy, we will analyze where context helps or hurts and report that result.

Acknowledgments. We thank Professor Lesia Semenova for feedback on the research question and evaluation. We used OpenAI Codex for coding, debugging, and editing assistance. Our team is responsible for checking the implementation, running the experiments, and interpreting the results.

References

[1] R. Hu et al. Benchmarking LLMs for fine-grained code review with enriched context in practice. *arXiv:2511.07017*, 2025. `https://arxiv.org/abs/2511.07017`.

[2] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma. Repoformer: Selective retrieval for repository-level code completion. *ICML*, PMLR 235:53270–53290, 2024.

[3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv:2305.14314*, 2023.